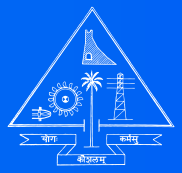
C2SAFERUST-LLM: INTERPROCEDURAL OWNERSHIP ANALYSIS FOR SAFE C-TO-RUST TRANSPILATION

GOKULKRISHNA A K
MCA
TCR25MCA-2028

Mini Project

Guided By:
Raseek C

Department of Computer Applications
Government Engineering College, Thrissur, Kerala



Outline

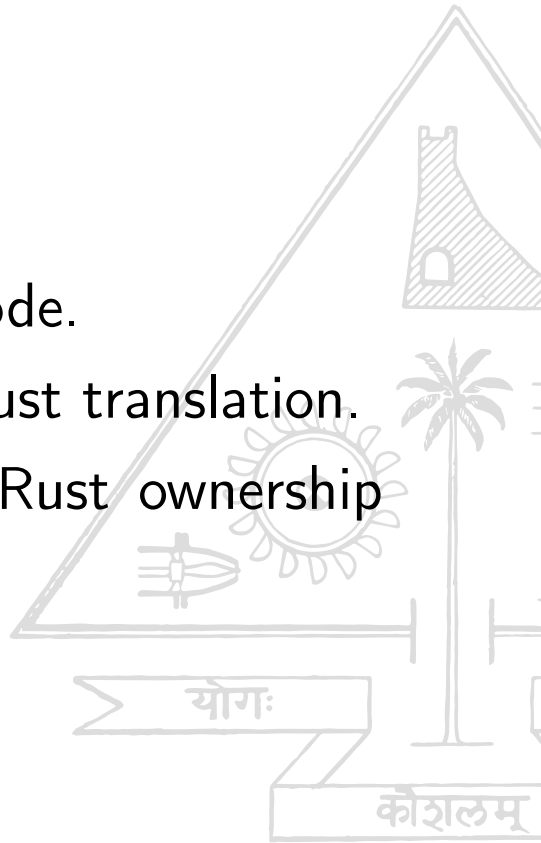
1. Introduction
2. Literature Review
3. Existing Systems
4. Proposed System
5. Requirements Specification
6. Input and Output
7. System Architecture & DFDs
8. System Modules
9. References

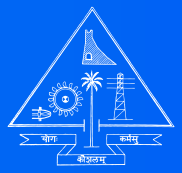




Introduction

- C is powerful but vulnerable to memory errors.
- Rust ensures memory safety at compile time.
- Safety issues mainly arise in `unsafe` and FFI code.
- Safe ownership inference is essential for C-to-Rust translation.
- This motivates translators that infer genuine Rust ownership rather than emit unsafe-laden mappings.





Introduction: Motivation

- **Predecessor system: C2SafeRust**
  - **Compiler-in-the-loop**: C → AST → alias/allocation model → 8 ownership categories → candidate Rust → rustc's JSON diagnostics as repair spec.
  - **Baseline** (intra-procedural, 10-program benchmark): **80%** compilation success, zero unsafe blocks in **75%** of successes, avg. **2.3** repair iterations.
  - Intra-procedural analysis misses ownership relationships across function calls.

Introduction: Motivation

pointer_analyzer.rs — intra-procedural view

SCOPE: PER-FUNCTION ONLY

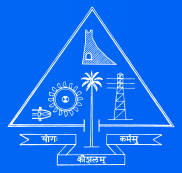
```c
// C source: idiom 1 — aliased mutable parameters
void update(int *a, int *b) {
    *a = 10;
    *b = 20;
}

void baz(void) {
    int x = 1;
    update(&x, &x);  // x == 20 afterward
}
```

POINTER CHAIN ANALYSER sees only `update`'s body → **a** and **b** look independent → emits **fn update(a: &mut i32, b: &mut i32)**. The alias only exists at the call site inside **baz**, which this per-function pass never opens.

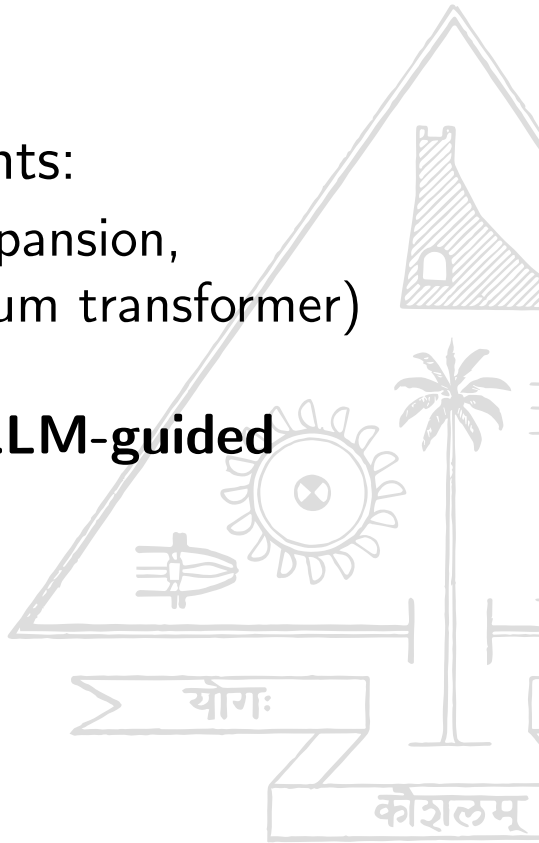
```
$ cargo build
error[E0499]: cannot borrow `x` as mutable more than once at a time
  --> src/pointer_analyzer.rs:9:20
   |
 9 |     update(&mut x, &mut x);
   |         ------  ^^^^^^ second mutable borrow occurs here
   |         |
   |         first mutable borrow occurs here
   = help: resolvable only with an interprocedural, call-site-aware alias check
```

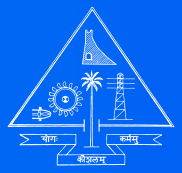
Figure 1: Pointer Chain Analyser, scoped per-function: `update(&x, &x)` looks safe inside `update` alone; the alias only exists at the call site in `baz`, so `rustc` rejects the naive translation with E0499.



Introduction: Objectives

- Extend C2SafeRust with five bundled components:
  - Widen the accepted C subset (preprocessor expansion, multi-level pointer chain analyser, union-to-enum transformer)
  - Add an **interprocedural analysis pass**
  - Replace the rule-based repair engine with an **LLM-guided repair agent**.

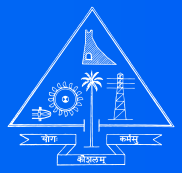




Literature Review

S. Klabnik and C. Nichols, "The Rust Programming Language," 2nd ed., *No Starch Press*, 2023.

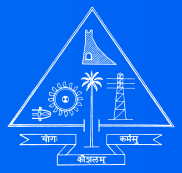
- Standard reference on how Rust's borrow checker eliminates data races and use-after-free at compile time, at no runtime cost.
- Establishes the appeal of Rust's ownership and borrowing model for systems software.
- **Observation:** Documents the guarantee's mechanism, but not where translation-time ownership evidence should come from.



Literature Review

N. D. Matsakis and F. S. Klock, "The Rust language," *ACM SIGAda Annual Conf. (HILT)*, 2014.

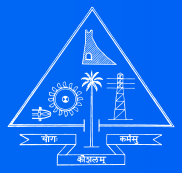
- First formalisation of Rust's ownership and borrowing model as a static alternative to garbage collection and manual memory management.
- Frames ownership as a type-system property, the theoretical foundation later built on by translation tooling.
- **Observation:** Foundational to the language design, but predates any discussion of automated translation tooling.



Literature Review

B. Qin, Y. Chen, Z. Yu, L. Song, and Y. Zhang, "Understanding memory and thread safety practices and issues in real-world Rust programs," *Proc. 41st ACM SIGPLAN Conf. Program. Lang. Des. Implement. (PLDI)*, 2020.

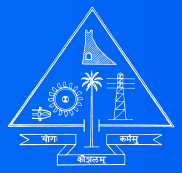
- Empirical study of memory- and thread-safety practices across published Rust crates.
- Finds violations concentrate at `unsafe` boundaries and FFI – safety issues migrate there rather than disappear.
- **Observation:** Every C pointer pattern a translator can't classify confidently becomes such a bypass – minimising these is safety-relevant, not cosmetic.



Literature Review

M. Emre, R. Schroeder, K. Dewey, and B. Hardekopf, "Translating C to safer Rust," *Proc. ACM Program. Lang.*, vol. 5, no. OOPSLA, 2021.

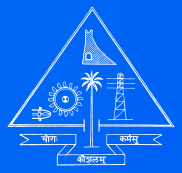
- Introduces Laertes, which treats rustc's diagnostics as a specification and iteratively rewrites raw-pointer types in response.
- This compiler-feedback strategy is the same one the base C2SafeRust pipeline builds on.
- **Observation:** Effective where a pointer's type is decidable within its own function – but limited exactly where a per-function view runs out.



Literature Review

M. Emre, P. Boyland, A. Parekh, R. Schroeder, K. Dewey, and B. Hardekopf, "Aliasing limits on translating C to safe Rust," *Proc. ACM Program. Lang.*, vol. 7, no. OOPSLA1, 2023.

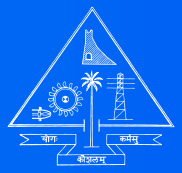
- Empirical follow-up measuring how far aliasing analysis alone can resolve raw pointers to safe references.
- Traces residual imprecision to alias equivalence classes that only become decidable once caller and callee are analysed together.
- **Observation:** An empirical demonstration, from inside a working system, of the interprocedural ownership-transfer gap.



Literature Review

J. Hong and S. Ryu, "Don't write, but return: Replacing output parameters with algebraic data types in C-to-Rust translation," *Proc. ACM Program. Lang.*, vol. 8, no. PLDI, 2024.

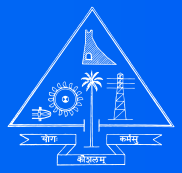
- Replaces C's output-parameter idiom (writing a result through a pointer argument) with an algebraic type such as `Option` or `Result`.
- Explicitly interprocedural: the transform only makes sense once the analysis sees what callers do with the written value.
- **Observation:** Resolves one of six target idioms (ownership created through a `T**` out-parameter), but does not generalise to the other five.



Literature Review

H. Peng, B. Kasikci, G. L. Bernstein, and M. D. Ernst, "Hayroll: A modular wrapper for translating C macros and conditional compilation to Rust," *Proc. ACM Program. Lang.*, vol. 10, no. PLDI, 2026.

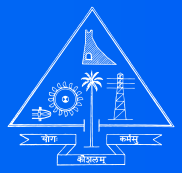
- Recovers macro-heavy C constructs at the preprocessor level, translating macros and conditional compilation to native Rust attributes.
- Widens the parseable C subset, operating strictly before semantic analysis.
- **Observation:** Complementary rather than competing – it does not address ownership inference across multiple interacting functions.



Literature Review

W. Yang, L. Song, and Y. Xue, "Rust-lancet: Automated ownership-rule-violation fixing with behavior preservation," *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng. (ICSE)*, 2024.

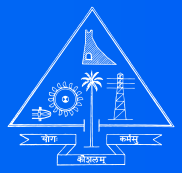
- Applies iterative, template-based repair (e.g. inserting `.clone()`) directly to already-broken Rust, validated by a behaviour-preservation check.
- A safety net operating after code generation, not before.
- **Observation:** Valuable, but repairing an already-broken translation is strictly more expensive than avoiding the mistake upstream.



Literature Review

M. Ling, Y. Yu, H. Wu, Y. Wang, J. R. Cordy, and A. E. Hassan, "In Rust we trust: A transpiler from unsafe C to safer Rust," *Proc. IEEE/ACM 44th Int. Conf. Softw. Eng.: Companion Proc.*, 2022.

- Applies a fixed catalogue of source-to-source rewrite rules, without compiler feedback, to reduce `unsafe`-marked function signatures.
- Operates on the output of a syntactic transpiler rather than on ownership semantics.
- **Observation:** Another post-hoc cleanup strategy sharing Rustlancet's limitation – it accepts ownership mistakes as a cost of doing business.

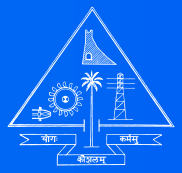


Literature Review: Summary

Author	Approach	Brief Observation
Klabnik & Nichols (2023)	Rust Language Reference	Documents borrow-checker guarantees, but not where translation-time ownership evidence comes from.
Matsakis & Klock (2014)	Ownership Model Formalisation	Foundational static ownership theory; predates translation tooling.
Qin et al. (2020)	Rust Safety Study	Shows safety violations concentrate at unsafe/FFI boundaries in real Rust code.
Emre et al. (2021)	Compiler-Feedback Translation	Effective intra-procedurally; limited exactly where per-function view runs out.
Emre et al. (2023)	Aliasing-Limits Study	Empirically traces residual imprecision to call-unaware alias reasoning.

Literature Review: Summary (contd.)

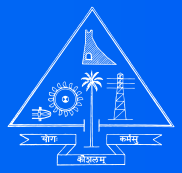
Author	Approach	Brief Observation
Hong & Ryu (2024)	Output-Parameter Transform	Resolves one idiom (T** out-param) inter-procedurally; does not generalise.
Peng et al. (2026)	Preprocessor Recovery	Widens parseable C subset; does not address ownership inference.
Yang et al. (2024)	Post-hoc AST Repair	Repairs broken translations after the fact; costlier than avoiding the mistake.
Ling et al. (2022)	Fixed Rewrite Catalogue	Reduces unsafe-marked signatures without compiler feedback.



Existing Systems

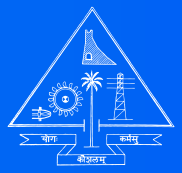
Existing systems focus on specific aspects of C-to-Rust translation rather than a complete solution.

- **C2SafeRust**:
  - Compiler-in-the-loop C-to-Rust pipeline.
  - Analyses strictly per function.
  - No interprocedural ownership tracking.
- **Laertes**:
  - Diagnostics-guided pointer rewriting.
  - Rule-based, per-function scope.
  - Misses call-site aliasing.
- **Hayroll**:
  - Macro/preprocessor recovery wrapper.
  - Widens parseable C subset.
  - No ownership inference.



Proposed System

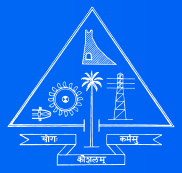
- **C2SafeRust-LLM** bundles five components into a single extension of the base pipeline:
  - A **preprocessor expansion** stage.
  - A **multi-level pointer chain analyser** (tracks `T**` and deeper).
  - A **union-to-enum transformer**.
  - A new **interprocedural analysis pass**.
  - An **LLM-guided repair agent**, replacing the rule-based `RepairEngine`.
- **In scope:** call-graph construction, interprocedural alias analysis, ownership-transfer summaries linking callee write-through pointers to caller locals.
- **Out of scope:** making idiom 6 fully static – routed to a conservative `Option<Box<T>>` fallback or the LLM repair agent.



Requirements Specification

Hardware Requirements

- **Processor**: Intel® Core™ i5 or AMD Ryzen™ 5 or higher
- **Memory**: Minimum 8 GB RAM; 16 GB recommended, since the call graph, alias summaries, and ownership maps must be held together
- **Storage**: 50–100 GB free disk space (Rust toolchain, cargo artifacts, benchmarks)
- **Network**: required when the repair loop escalates to the LLM repair agent's API

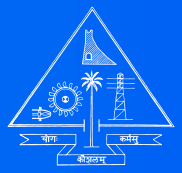


Requirements Specification

Software Requirements

- **Operating System**: Ubuntu 20.04 LTS or later, Windows 10/11, or macOS
- **Programming Language**: Rust (stable channel)
- **Build Tool**: Cargo
- **Version Control**: Git
- **Required Rust Crates**:
  - **serde** – deserialises rustc's JSON diagnostics
  - **petgraph** – call graph and alias graph
  - **reqwest** – HTTP client to the LLM repair agent
  - **egui** – GUI for source, generated Rust, and report

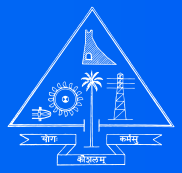




Requirements Specification

Software Requirements for Development

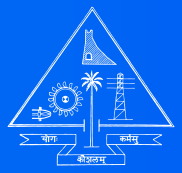
- **Operating System**: Ubuntu 20.04+, Windows, or macOS
- **Back End**: Rust (stable channel)
- **IDE Used**: Visual Studio Code with the rust-analyzer extension
- **Version Control**: Git
- **Compiler Toolchain**: rustc, invoked with `--error-format=json`, used both to validate translations and to drive the LLM-guided repair loop



Requirements Specification

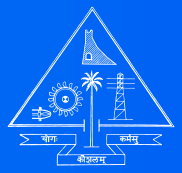
Functional Requirements

- **Input Validation:** Accept a single C source file, verify it is readable and correctly encoded, and reject malformed input with a descriptive error.
- **Preprocessor Expansion:** Resolve macros and conditional-compilation directives before AST construction, widening the parseable C subset.
- **AST Construction:** Parse the validated input into an internal AST shared by semantic analysis, interprocedural analysis, and code generation.
- **Multi-Level Pointer Chain Analysis:** Track pointer chains of depth two or greater, classifying whether each level is freed, reassigned, or newly allocated.



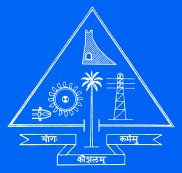
Requirements Specification

- **Union-to-Enum Transformation:** Identify C unions with a discriminable active-variant pattern and transform them into safe Rust enums.
- **Rust Code Generation:** Generate candidate Rust from the AST, semantic model, and ownership classifications, reusing existing type-mapping rules.
- **Call Graph Construction:** Build a call graph with one node per function and one edge per call site.
- **Interprocedural Alias Analysis:** Determine, per call site, whether two pointer arguments can refer to the same storage.
- **Ownership-Transfer Summaries:** Compute whether a `T**`-taking function frees/allocates its pointee, propagated to every call site.



Requirements Specification

- **Cross-Function Union Tracking:** Extend union flow analysis to an interprocedural reaching-definitions analysis across function boundaries.
- **Data-Dependent Ownership Fallback:** Detect run-time-dependent ownership and emit a conservative `Option<Box<T>>` translation or escalate to the LLM agent.
- **LLM-Guided Repair Agent:** Replace the rule-based `RepairEngine` with an agent that proposes patches from rustc diagnostics and re-verifies by recompilation.



Requirements Specification

Non-Functional Requirements

- **Scalability:** Summaries computed once per function and reused across call sites, so cost grows with call-graph size, not call-site count.
- **Backward Compatibility:** Programs without any of the six target idioms translate identically to the existing system.
- **Transparency:** Every changed ownership decision is recorded in the `TranslationReport`, tagged with its triggering idiom.
- **Maintainability:** A distinct, independently testable module consuming `SemanticModel` and feeding the existing `OwnershipInferrer`



Input and Output

- **Input:** A single C source file (.c)
- **Input Format:** Syntactically valid C source code, restricted to a single translation unit.
- **Output:**
  - **Generated Rust (.rs):** Rust code with inferred ownership.
  - **Translation Report:** Summary of ownership decisions and repairs.
  - **Repair Log:** `rustc` diagnostics and applied fixes.
  - **GUI (egui):** Interactive view of source, Rust output, and report.

System Architecture

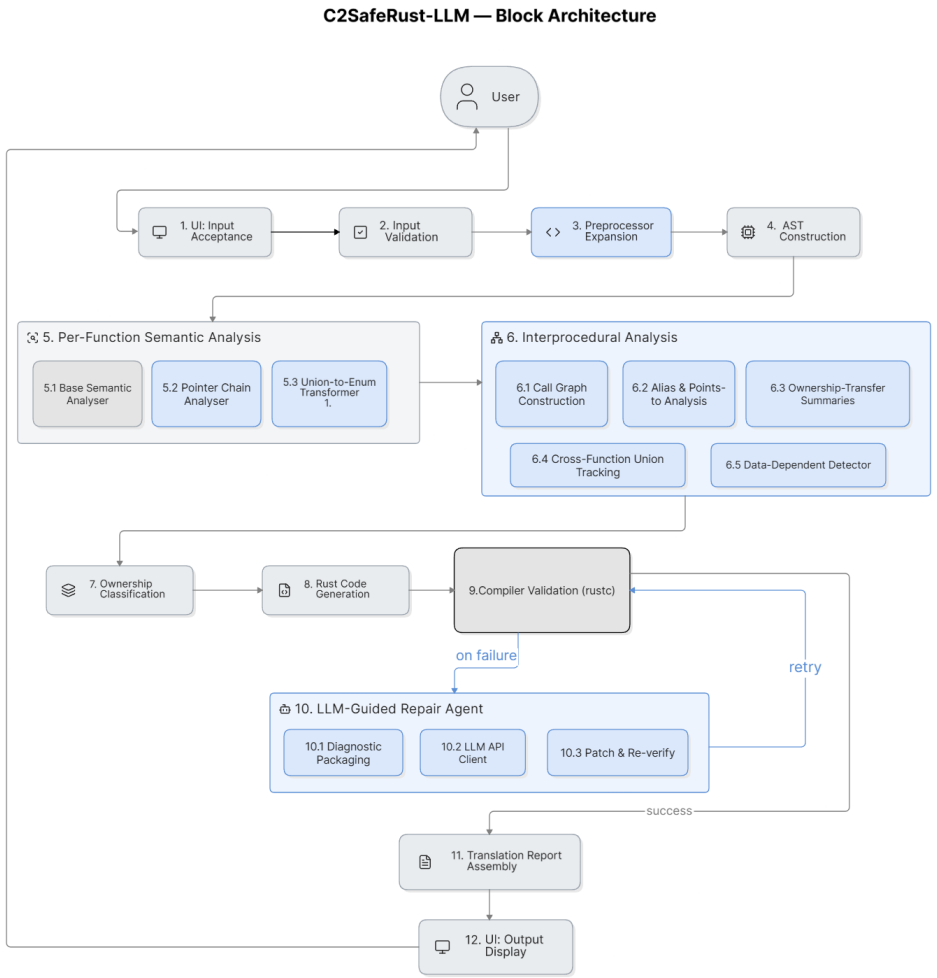


Figure 2: C2SafeRust-LLM — Block Architecture

Data Flow Diagram: Level 0

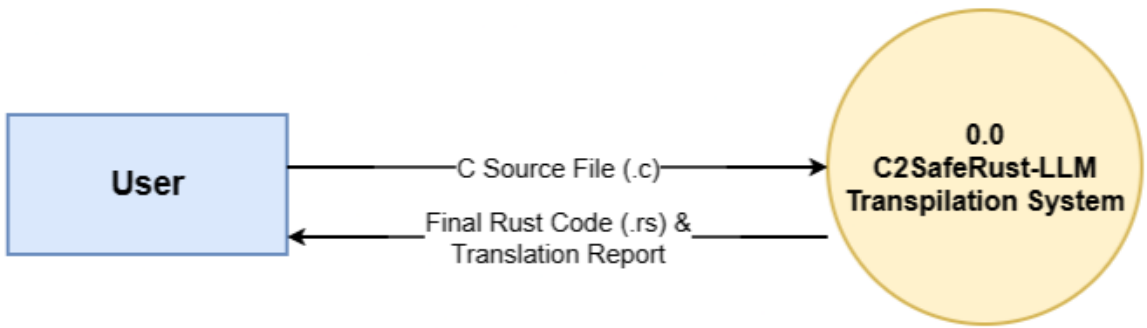


Figure 3: Level 0 DFD — Context Diagram

Data Flow Diagram: Level 1

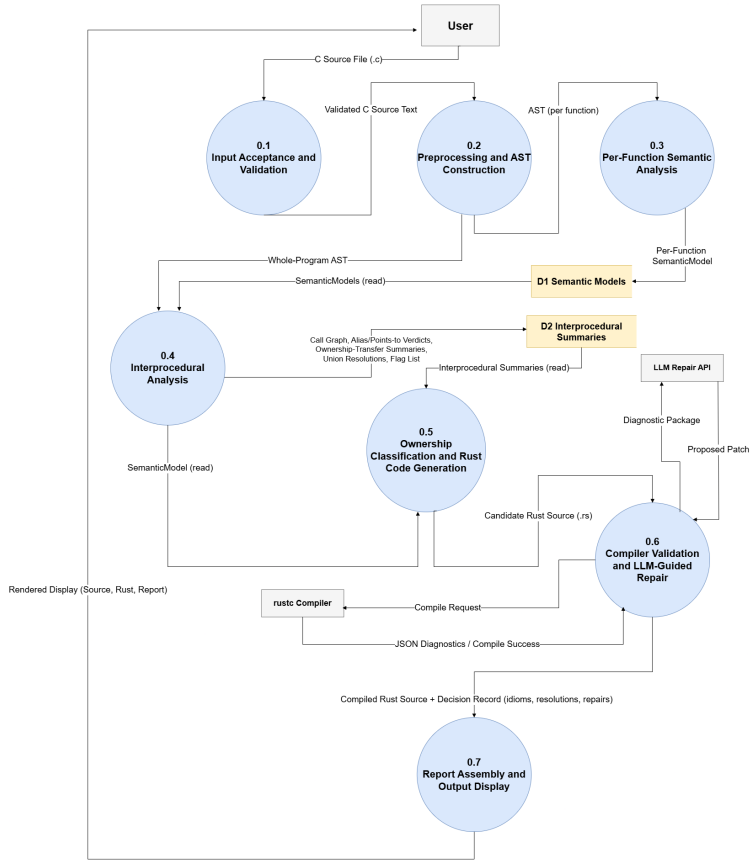


Figure 4: Level 1 DFD — Pipeline Decomposition

Data Flow Diagram: Level 2 (Process 0.3)

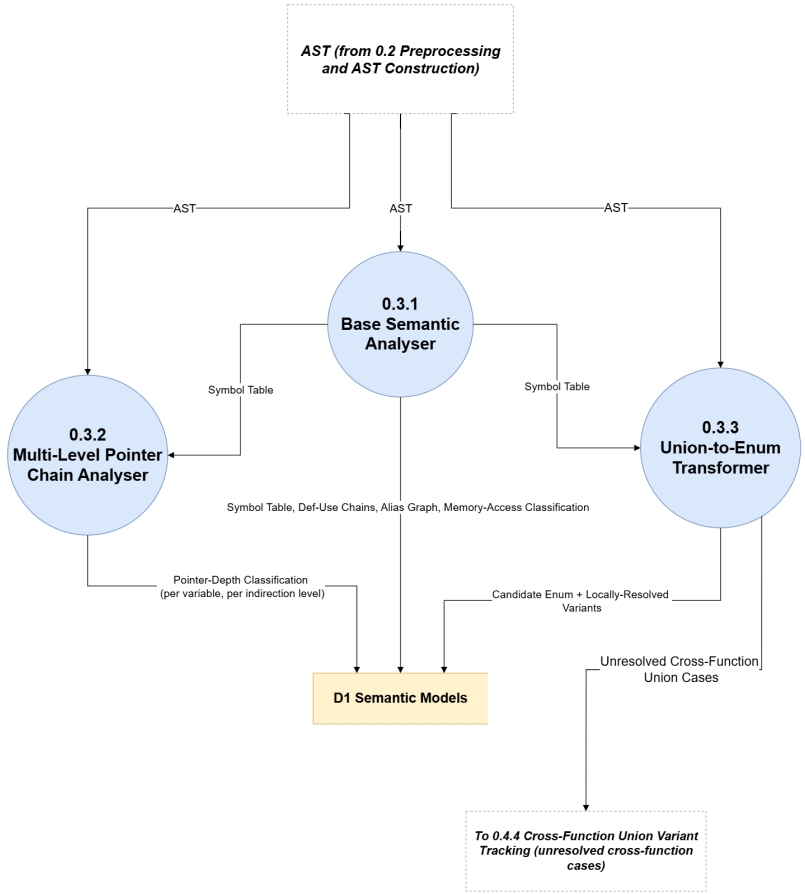


Figure 5: Expansion of Process 0.3: Per-Function Semantic Analysis

Data Flow Diagram: Level 2 (Process 0.4)

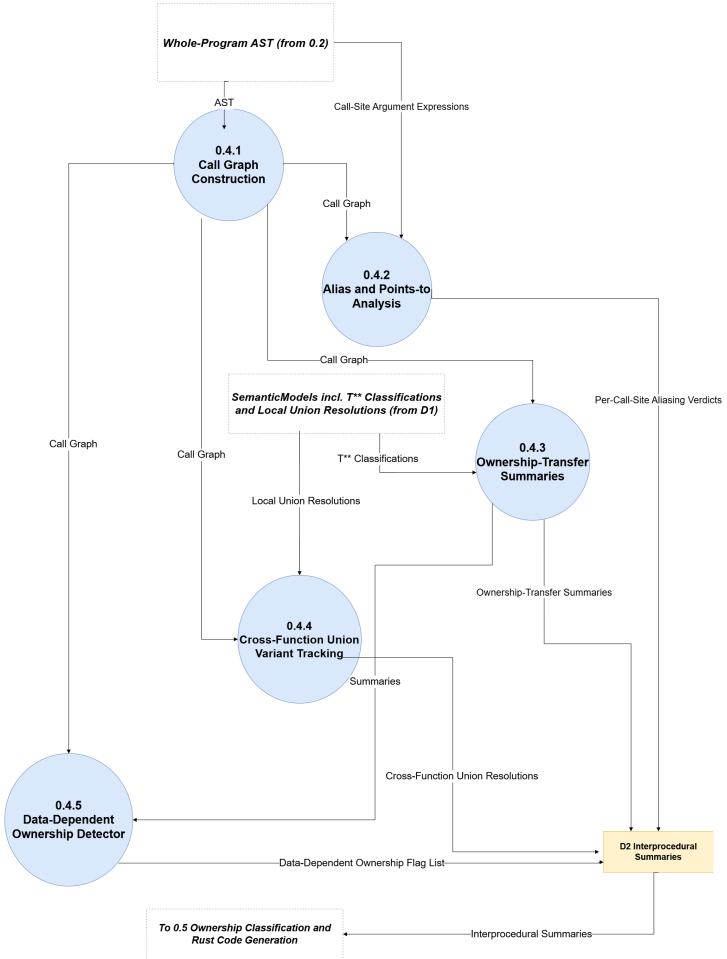


Figure 6: Expansion of Process 0.4: Interprocedural Analysis

Data Flow Diagram: Level 2 (Process 0.6)

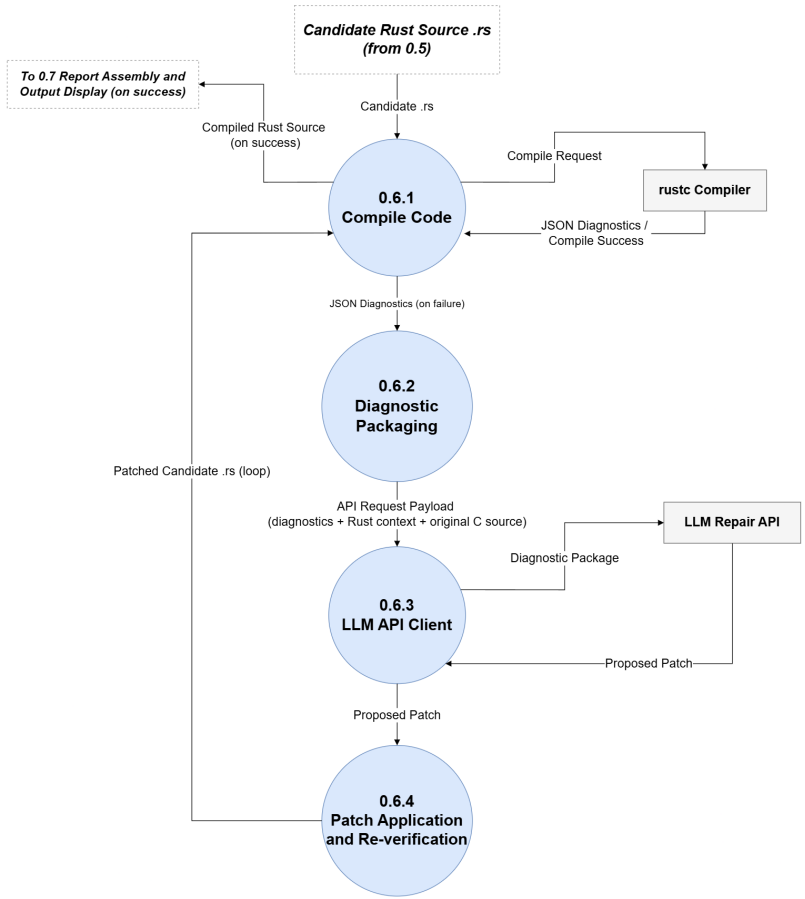
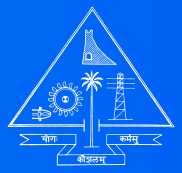
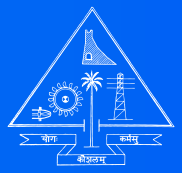


Figure 7: Expansion of Process 0.6: Compiler Validation & LLM Repair



System Modules

- **Code Acceptance & Validation:** Handles CLI/GUI inputs, pre-flight syntax checks, and complexity scoring.
- **Preprocessing & AST Construction:** Expands macros, parses C source into location-aware ASTs.
- **Per-Function Semantic Analysis:** Builds def-use chains, alias graphs, T^{**} pointer classifications, and local union transformations (**D1**).
- **Interprocedural Analysis:** Constructs global call graphs, resolves cross-function aliasing, derives ownership-transfer summaries, and tracks data dependencies (**D2**).



System Modules

- **Ownership Classification & Code Gen:** Maps 7 `OwnershipKind` variants to safe Rust, emits struct derives, and formats candidate Rust code.
- **Compiler Loop & LLM Repair:** Submits code to `rustc`, captures JSON diagnostics, packages requests, and applies proposed LLM patches iteratively.
- **Report Assembly & Output Display:** Compiles decision logs, renders side-by-side GUI source views, and displays execution comparison outputs.



References

[1] S. Klabnik and C. Nichols, "The Rust Programming Language," 2nd ed. San Francisco, CA, USA: No Starch Press, 2023.

[2] B. Qin, Y. Chen, Z. Yu, L. Song, and Y. Zhang, "Understanding memory and thread safety practices and issues in real-world Rust programs," in *Proc. 41st ACM SIGPLAN Conf. Program. Lang. Des. Implement. (PLDI)*, 2020, pp. 763–779.

[3] N. D. Matsakis and F. S. Klock, "The Rust language," in *Proc. ACM SIGAda Annual Conf. (HILT)*, 2014, pp. 103–104.

[4] M. Emre, R. Schroeder, K. Dewey, and B. Hardekopf, "Translating C to safer Rust," *Proc. ACM Program. Lang.*, vol. 5, no. OOPSLA, Art. no. 121, Oct. 2021.



References

[5] M. Emre, P. Boyland, A. Parekh, R. Schroeder, K. Dewey, and B. Hardekopf, "Aliasing limits on translating C to safe Rust," *Proc. ACM Program. Lang.*, vol. 7, no. OOPSLA1, Art. no. 94, pp. 551–579, Apr. 2023.

[6] J. Hong and S. Ryu, "Don't write, but return: Replacing output parameters with algebraic data types in C-to-Rust translation," *Proc. ACM Program. Lang.*, vol. 8, no. PLDI, Art. no. 176, Jun. 2024.

[7] H. Peng, B. Kasikci, G. L. Bernstein, and M. D. Ernst, "Hayroll: A modular wrapper for translating C macros and conditional compilation to Rust," *Proc. ACM Program. Lang.*, vol. 10, no. PLDI, Art. no. 198, Jun. 2026.



References

[8] W. Yang, L. Song, and Y. Xue, "Rust-lancet: Automated ownership-rule-violation fixing with behavior preservation," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng. (ICSE)*, 2024, pp. 85:1–85:13.

[9] M. Ling, Y. Yu, H. Wu, Y. Wang, J. R. Cordy, and A. E. Hassan, "In Rust we trust: A transpiler from unsafe C to safer Rust," in *Proc. IEEE/ACM 44th Int. Conf. Softw. Eng.: Companion Proc. (ICSE-Companion)*, 2022, pp. 354–355.



Thank You